

Introduction to Unix/Linux: OS System Scavenger Hunt

You should work together with a partner on all in class activities and have discussions, but each person should work on their own computer, and complete their own worksheet.

Learning Objectives: *Be able to execute basic unix commands and navigate around the operating system using a Terminal.*

1. **Command line interface.** The Mac Operating System (MacOS) is Unix-based; this means we can use the usual command line options that work in a (purely) Unix/Linux environment. If you have been using a Microsoft-based laptop you'll have to use one of the classroom Macs for this exercise. You will access these command lines via a "terminal", which is a window where you can input commands and view text results.

- Use the spotlight search (cmd+space) to open *Terminal*.

2. **Basic Commands.** Let's orient ourselves using some command commands. Enter the following commands in the Terminal to find out where (and who) you are.

- Enter the following commands in the *Terminal* to find out where (and who) you are. Record the command output. What does each do?

- whoami

- pwd

- ls [list a couple of the files, do these make sense?]

- ls ../ [what does the ../ mean?]

- date

- uname -a

- w

- Lets check your guesses! You can access the manual for any command via the command `man`. First, type `man man` (use 'q' to quit). What did that do? What did you find?

- Next do a `man` on each of the commands above to learn what they truly do. Which of your guesses were right, and which were wrong?

- Make sure to record the location of your home directory (the directory the terminal first accessed) for later.

3. **File information.** It is possible to learn some information about files straight from the terminal without opening (or right-clicking on) them.

- Pick one of the files in the current directory and enter: `du -h name-of-file`. *What does the command `du` do?*

- The `-h` after the command `du` is what's known as a "flag". What did the `-h` flag do? Take a look at the `du manual`, i.e., type: `man du` to confirm your guess. *According to the manual, what does the `-h` flag do?*

4. **Directory Tree.** Files are organized hierarchically in the system, i.e., files exist in a series of "folders" and sub-folders. Let's look at the system starting with the highest level directory, which is known as the "root" directory. To access the root directly, you would use the file path (or path for short) /

- Type: `ls /` *What directories do you see? Do any look familiar? Where is your home directory relative to root?*

- Use `ls` to look at the contents of the directory `/bin`. *What is stored there?* Try typing `/bin/pwd`. *What happens?*

5. **Navigating the directory tree.** You are currently located in a particular directory of the tree (usually wherever your user home directory is). You can change your location in the tree, i.e., navigate between folders using the `cd` (change directory) command: `cd path-to-where-you-want-to-go`

- Use `cd` to move to the `/Applications` folder. *What are some things stored there? Does this make sense?*

6. File information. Let's investigate more file information.

- Pick one and type: `ls -l name-of-file`. This should return information about who installed or created it (the file 'owner') and when. *Based on what is returned who installed this file and when?* (More information about the `-l` flag can be contained by typing: `man ls`.)

7. File permissions. The first item returned by '`ls -l`' is a 10 character string. The first character indicates whether the item is a directory ('`d`') or a file ('`-`'). The next three characters give the *file owner* permissions (e.g. root), the next 3 give the *group* permissions and the last three give the *world* (you and all other users, i.e., everyone) permissions. To generalize permissions, users can be assigned to different groups, where everyone in the group is given the same access permissions. Here, the letter '`r`' = read permission (you can open and look at it), '`w`' = write permission (the file can be edited) and '`x`' = execute permissions (can be run).

- What does `drwxr- - r - -` mean?

- Look at the permissions of some of the Applications — what do you find? Who tends to have what kind of access to the Applications?

8. Changing file properties. Return to your home directory (you can do this by just typing `cd`; pro tip - typing "`cd -`" will take you back to where you came from). Lets explore how to change a file name. We'll do this using the file "`testfile.txt`", which you can download from today's module on Canvas. We can use Unix commands to do this on the command line, as well as changing the file's permissions!

- Chances are this file was not saved to your home directory. Navigate to wherever it is. (e.g., if its on your desktop: `cd Desktop`).
- Make a new copy of this file with a new name using the `cp` command: `cp testfile.txt newfile.txt`

- Check the permissions of the new file. *What are the default file permissions? Who is the owner?*

- You can make it writable for everyone (editable) by typing: `chmod a+w filename-of-file`. Here, `a` = all, and `w` means writable. Alternatively, `chmod g+w` makes it only group writable (`g` = group); `chmod u+x` makes it executable by the owner. Changing the `+` to a `-` removes a type of permission. (There is also a way to do this with `chmod` and numbers — see the file permissions section of the reference sheet). Go ahead and change the permissions. (Note: if you are not the file owner you will not be able to change the permissions.)

9. File Quick-look. Text files may be inspected in the Terminal without using any particular application using the `head` (first X lines, where X=10 by default), `tail` (last X lines, where X=10 by default) and `more` (scrolls through the file starting with whatever lines fit in the terminal window) commands.

- Take a quick look at the file content by using the `head`, `tail` and `more` commands. (Return or space will scroll down when using `more` and `q` will quit).

- Use the `man` command to figure out what flag to use for `head` to display more (or less) than 10 lines.

10. Deleting Files. There is a command to delete files and directories: `rm`. This should never be used lightly — unlike deleting files with your mouse, the file isn't simply moved to trash.

WARNING!!! There is no undo. Once you've deleted the file it's gone (unless you have a system backup).

- Type: `rm testfile.txt` to delete the old file.

11. Locating files. Some files and applications are accessible from anywhere in the system. In Unix/Linux this is an intentional choice, i.e., their locations are intentionally specified to be accessible. We say they are “in your path.” If something is in a directory in your path, you can use the `which` command (`which name-of-application`) to figure out where it is located. Files located in any directory “in your path” are accessible from anywhere (remember above how we found out that `pwd` is located in `/bin`, yet you can access the `pwd` command anywhere?).

- Print your path by typing: `echo $PATH`. *What do you find is in your path?*

- Use the `which` command to figure out where `jupyter-notebook` lives (we'll use this program a lot; the `which` command is particularly important if you have multiple versions of some application, such as Python).

12. **New Directories.** The command `mkdir` will make a new file directory.

- Use the `mkdir` command to make a new directory in your home directory (you can print the location of your home directory using: `echo $HOME`).
- Copy (`cp file-name-to-be-copied directory-where-its-going`) or move (`mv file-name-to-be-copied directory-where-its-going`) a few of your files (from the desktop, downloads, home directory etc) into this directory. (WARNING: Make sure none of the files you move have the same name: the old one will be overwritten by the new one you move to the directory without any warning. Likewise make sure you don't `mv` one file onto another — the second one (the one at the destination) will be overwritten; this is NOT undoable and there is no warning).

13. **Local Shell Files.** One of the most useful hidden files is the `.bashrc` (or `.zshrc`) profile file.

This file is in the home directory and specifies additional system settings and aliases, which are to be applied every time you log in. Should you wish to customize some settings, always use the local file `~/.bashrc` located in your home directory. It is best practice to first copy the contents of your default `~/.bashrc` file to a separate file in case you ever accidentally delete some, or all, of its contents. Whenever you login or launch a new terminal, the contents of your `.bashrc` file are automatically sourced. If you change the content of the `.bashrc` file in the middle of a session, then you must activate the updated contents by sourcing the file using the `source` command in your home directory: `source ~/.bashrc`

14. **Alias.** It is possible to create shorter names for commands you find yourself frequently typing, or make shortcuts for commands with specific flags. These shortened names are called *aliases*. I strongly suggest aliasing your `"rm"` command to `"rm -i"`, which will confirm that you want to delete files. Add this to your `.bashrc` file: `alias rm='rm -i'`

15. **Experiment** with the other commands on the Unix Guide, provide in the Canvas module, especially `"gzip"`, `"gunzip"` and `"tar"`.